

EXPERIMENT 3

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Theory :

Blockchain is a distributed and decentralized digital ledger used to store transactions or other information in a sequence of interconnected blocks. Each block contains important information such as transaction data, timestamp, previous block's hash, and its own unique hash. The hash acts as a digital fingerprint of the block and helps maintain the integrity of the stored data.

Blocks are connected because each block stores the hash of the previous block. Therefore, if any information in a previous block is modified, its hash changes and the connection with the next block becomes invalid. This provides **data integrity and immutability**. Blockchain eliminates the need for a single central authority by maintaining copies of the ledger across multiple participating nodes.

A **cryptocurrency** is a digital form of currency that uses blockchain technology to record and verify transactions. Instead of depending on a centralized organization such as a bank, cryptocurrency transactions are recorded and validated by participating nodes in a blockchain network.

In this experiment, a cryptocurrency is simulated using **Python**. The blockchain consists of multiple nodes that maintain their own copies of the ledger. Transactions contain information about the **sender, receiver, and amount**. These transactions are initially kept as pending transactions and are later included in a block through the mining process.

Mining is the process of adding a new block containing pending transactions to the blockchain. In a Proof-of-Work based blockchain, the miner must solve a computational puzzle to find a valid proof.

The mining process consists of the following steps:

1. Pending transactions are collected.
2. Transactions are placed into a new block.
3. The miner performs the **Proof-of-Work (PoW)** computation.
4. A valid proof is found.
5. The new block is added to the blockchain.
6. The block can then be shared with connected nodes.
7. The miner receives a cryptocurrency reward.

Thus, mining provides a mechanism for adding new blocks and maintaining the blockchain network.

Proof-of-Work (PoW) is a consensus mechanism in which miners perform computational work to find a valid solution to a predefined mathematical problem. The miner repeatedly tries different values until a valid proof satisfying the required condition is obtained.

The purpose of Proof-of-Work is to make block creation computationally difficult and help prevent unauthorized or fraudulent modification of the blockchain. After a valid proof is found, the block can be added to the chain.

In this experiment, mining is implemented as part of the cryptocurrency blockchain, allowing students to understand how computational work is used to add blocks to a blockchain.

A **transaction** represents the transfer of cryptocurrency from one participant to another. Each transaction contains three basic fields:

- **Sender** – person or account sending the cryptocurrency.
- **Receiver** – person or account receiving the cryptocurrency.
- **Amount** – quantity of cryptocurrency being transferred.

Before transactions are added to a block, they remain in a collection of pending transactions commonly referred to as a **mempool**. The miner selects pending transactions from this pool and includes them in a new block.

For example: Sender: Alice
Receiver: Bob
Amount: 50

When the block is mined, the pending transactions are included in the block. A mining reward transaction can also be added to compensate the miner.

Multi-Node P2P Blockchain Network

A **Peer-to-Peer (P2P) network** allows blockchain nodes to communicate directly with one another without depending on a central server. In this experiment, three independent blockchain nodes are created:

Node 1 → Port 5001
Node 2 → Port 5002
Node 3 → Port 5003

Each node runs independently and maintains its own copy of the blockchain. The nodes are connected using the `/connect_node` API and can communicate with their peers to share and synchronize blockchain information.

Consensus and Longest Chain Rule

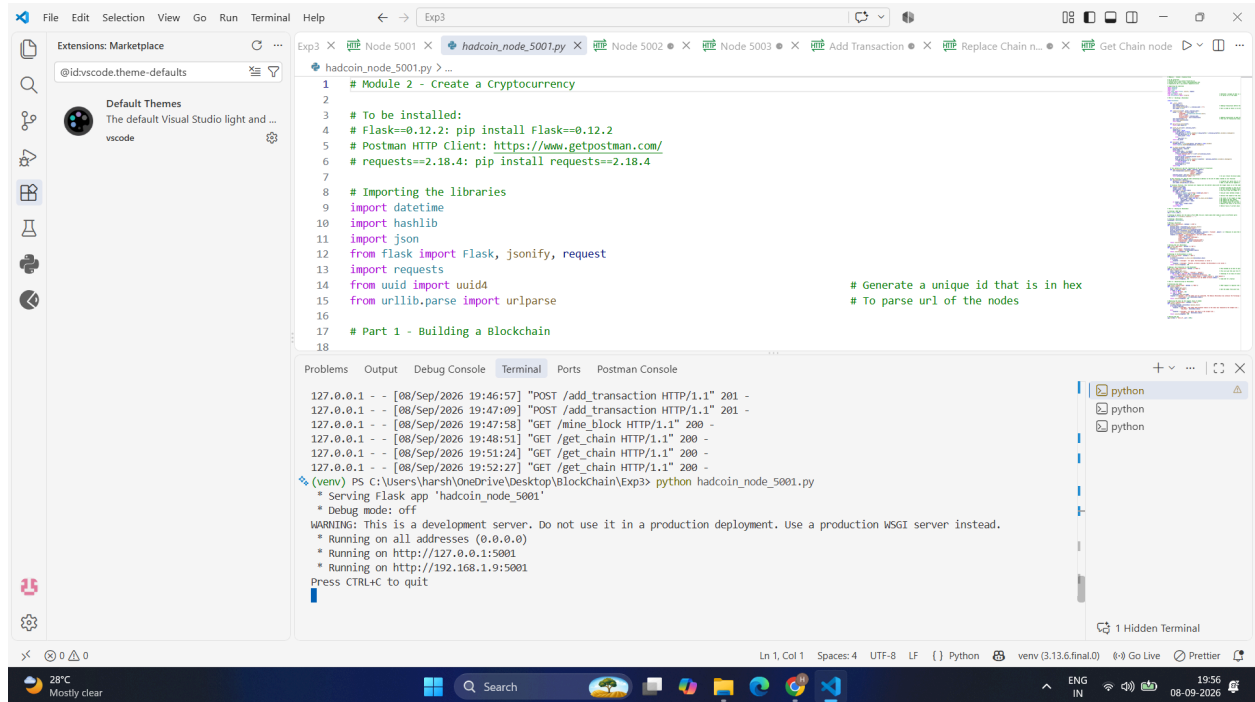
Since multiple nodes maintain separate copies of the blockchain, there can be situations where different nodes have different versions or lengths of the chain. A **consensus mechanism** is therefore required to ensure that nodes eventually agree on a common blockchain.

This experiment uses the **Longest Chain Rule**. According to this rule, when a node finds multiple valid blockchain versions, it selects the **longest valid chain** and replaces its current chain if necessary. This helps maintain consistency among the nodes.

The `/replace_chain` operation performs this synchronization. The node requests the chains from its peers, identifies a longer valid chain, and replaces its own chain with it when required.

Code and Output :

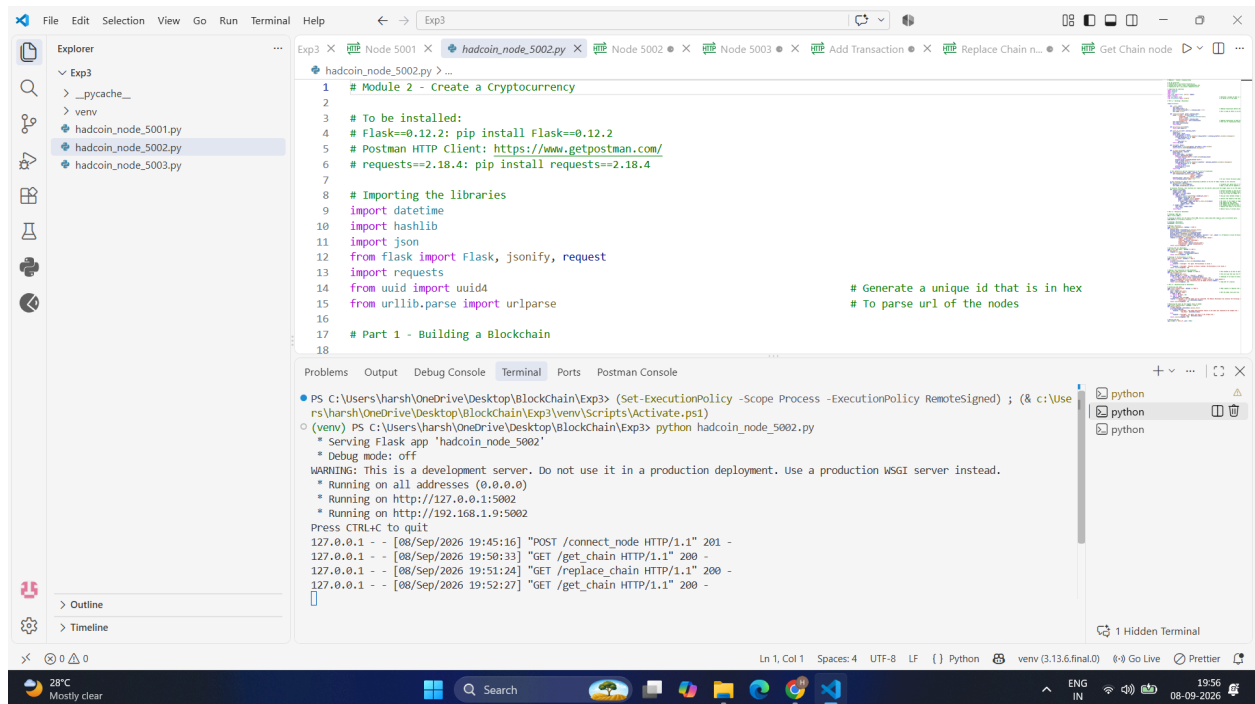
Step 1 : Three Nodes Running



This screenshot shows the Visual Studio Code interface with the 'Exp3' workspace. The Explorer sidebar on the left shows the file structure with three Python files: `hadcoin_node_5001.py`, `hadcoin_node_5002.py`, and `hadcoin_node_5003.py`. The main editor displays the code for `hadcoin_node_5001.py`, which includes comments for installing Flask, Postman, and requests, and imports for datetime, hashlib, json, flask, jsonify, request, uuid, and urllib.parse. The code is part of a module to create a cryptocurrency and build a blockchain. The Terminal panel at the bottom shows the command `python hadcoin_node_5001.py` being executed, resulting in a Flask app being served on `http://127.0.0.1:5001`. The status bar at the bottom indicates the file is in UTF-8 encoding and the Python interpreter is set to `venv (3.13.6.final.0)`.

```
1 # Module 2 - Create a Cryptocurrency
2
3 # To be installed:
4 # Flask==0.12.2: pip install Flask==0.12.2
5 # Postman HTTP Client: https://www.getpostman.com/
6 # requests==2.18.4: pip install requests==2.18.4
7
8 # Importing the libraries
9 import datetime
10 import hashlib
11 import json
12 from flask import Flask, jsonify, request
13 import requests
14 from uuid import uuid4
15 from urllib.parse import urlparse
16
17 # Part 1 - Building a Blockchain
18
```

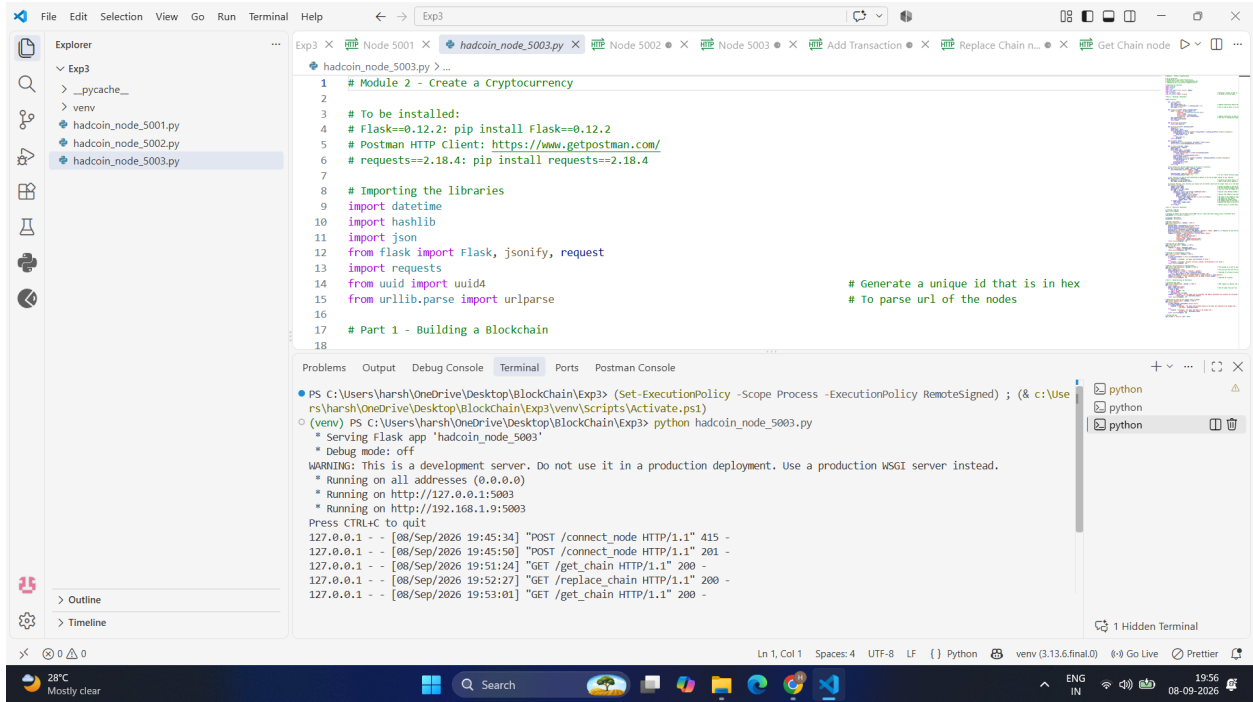
127.0.0.1 - - [08/Sep/2026 19:46:57] "POST /add_transaction HTTP/1.1" 201 -
127.0.0.1 - - [08/Sep/2026 19:47:09] "POST /add_transaction HTTP/1.1" 201 -
127.0.0.1 - - [08/Sep/2026 19:47:58] "GET /mine_block HTTP/1.1" 200 -
127.0.0.1 - - [08/Sep/2026 19:48:51] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [08/Sep/2026 19:51:24] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [08/Sep/2026 19:52:27] "GET /get_chain HTTP/1.1" 200 -
(venv) PS C:\Users\harsh\OneDrive\Desktop\Blockchain\Exp3> python hadcoin_node_5001.py
* Serving Flask app 'hadcoin_node_5001'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5001
* Running on http://192.168.1.9:5001
Press CTRL+C to quit



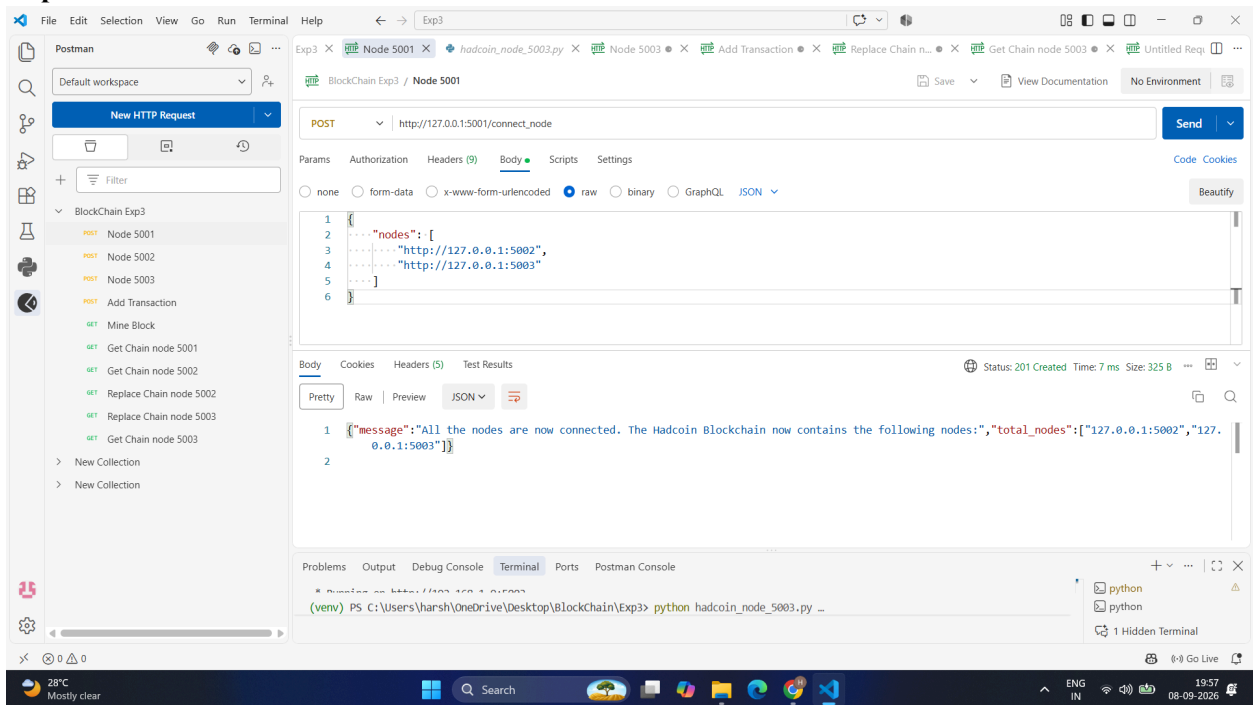
This screenshot shows the Visual Studio Code interface with the 'Exp3' workspace. The Explorer sidebar on the left shows the file structure with three Python files: `hadcoin_node_5001.py`, `hadcoin_node_5002.py`, and `hadcoin_node_5003.py`. The main editor displays the code for `hadcoin_node_5002.py`, which includes comments for installing Flask, Postman, and requests, and imports for datetime, hashlib, json, flask, jsonify, request, uuid, and urllib.parse. The code is part of a module to create a cryptocurrency and build a blockchain. The Terminal panel at the bottom shows the command `python hadcoin_node_5002.py` being executed, resulting in a Flask app being served on `http://127.0.0.1:5002`. The status bar at the bottom indicates the file is in UTF-8 encoding and the Python interpreter is set to `venv (3.13.6.final.0)`.

```
1 # Module 2 - Create a Cryptocurrency
2
3 # To be installed:
4 # Flask==0.12.2: pip install Flask==0.12.2
5 # Postman HTTP Client: https://www.getpostman.com/
6 # requests==2.18.4: pip install requests==2.18.4
7
8 # Importing the libraries
9 import datetime
10 import hashlib
11 import json
12 from flask import Flask, jsonify, request
13 import requests
14 from uuid import uuid4
15 from urllib.parse import urlparse
16
17 # Part 1 - Building a Blockchain
18
```

PS C:\Users\harsh\OneDrive\Desktop\Blockchain\Exp3> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Use
rs\harsh\OneDrive\Desktop\Blockchain\Exp3\venv\Scripts\Activate.ps1)
(venv) PS C:\Users\harsh\OneDrive\Desktop\Blockchain\Exp3> python hadcoin_node_5002.py
* Serving Flask app 'hadcoin_node_5002'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5002
* Running on http://192.168.1.9:5002
Press CTRL+C to quit
127.0.0.1 - - [08/Sep/2026 19:45:16] "POST /connect_node HTTP/1.1" 201 -
127.0.0.1 - - [08/Sep/2026 19:50:33] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [08/Sep/2026 19:51:24] "GET /replace_chain HTTP/1.1" 200 -
127.0.0.1 - - [08/Sep/2026 19:52:27] "GET /get_chain HTTP/1.1" 200 -



Step 2 : Nodes connected



Postman interface showing a REST client request to Node 5002.

Request: POST `http://127.0.0.1:5002/connect_node`

Body (raw):

```
1 {
2   "nodes": [
3     "http://127.0.0.1:5001",
4     "http://127.0.0.1:5003"
5   ]
6 }
```

Response (JSON):

```
1 {"message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:", "total_nodes": ["127.0.0.1:5001", "127.0.0.1:5003"]}
2
```

Terminal:

```
(venv) PS C:\Users\harsh\OneDrive\Desktop\BlockChain\Exp3> python hadcoin_node_5003.py ...
```

Postman interface showing a REST client request to Node 5003.

Request: POST `http://127.0.0.1:5003/connect_node`

Body (raw):

```
1 {
2   "nodes": [
3     "http://127.0.0.1:5001",
4     "http://127.0.0.1:5002"
5   ]
6 }
```

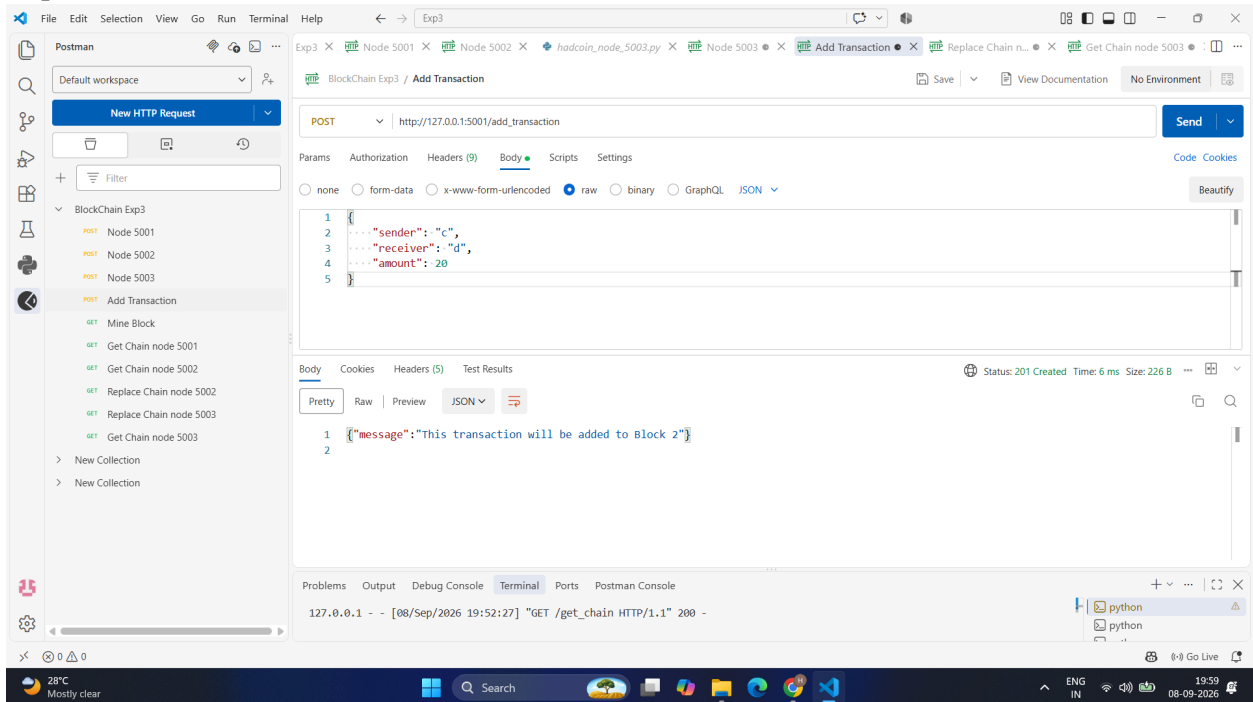
Response (JSON):

```
1 {"message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:", "total_nodes": ["127.0.0.1:5002", "127.0.0.1:5001"]}
2
```

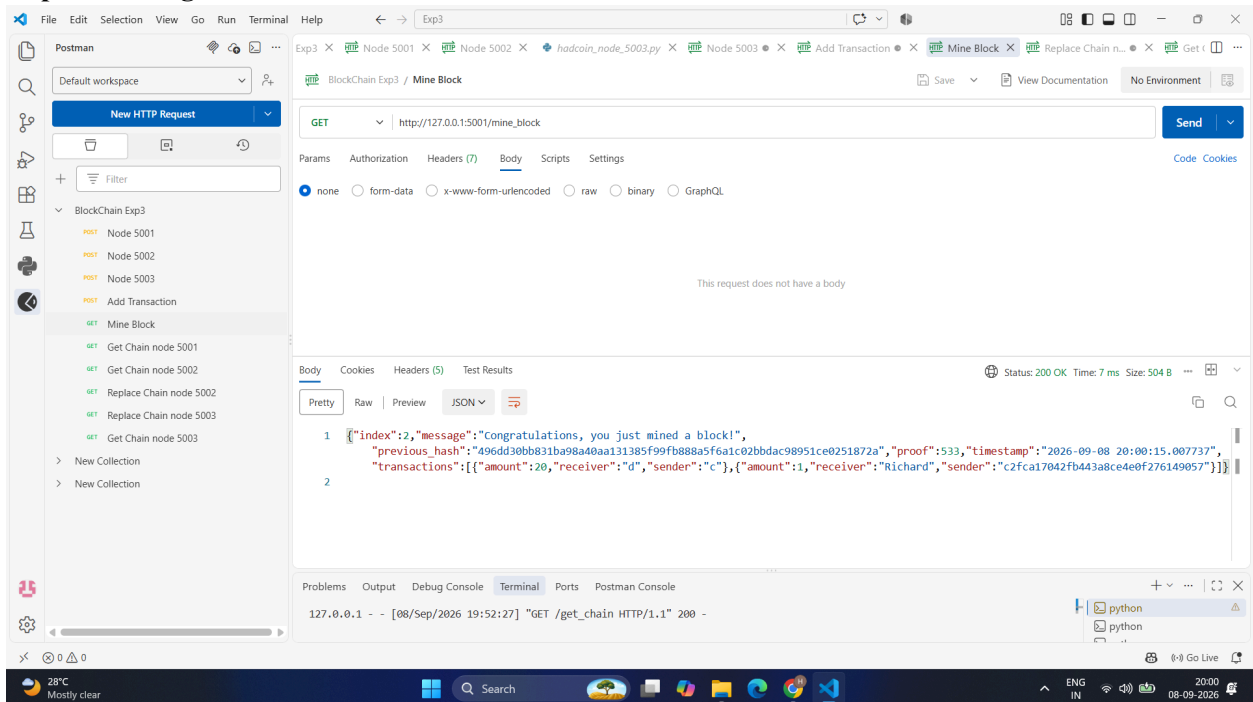
Terminal:

```
127.0.0.1 - - [08/Sep/2026 19:52:27] "GET /get_chain HTTP/1.1" 200 -
```

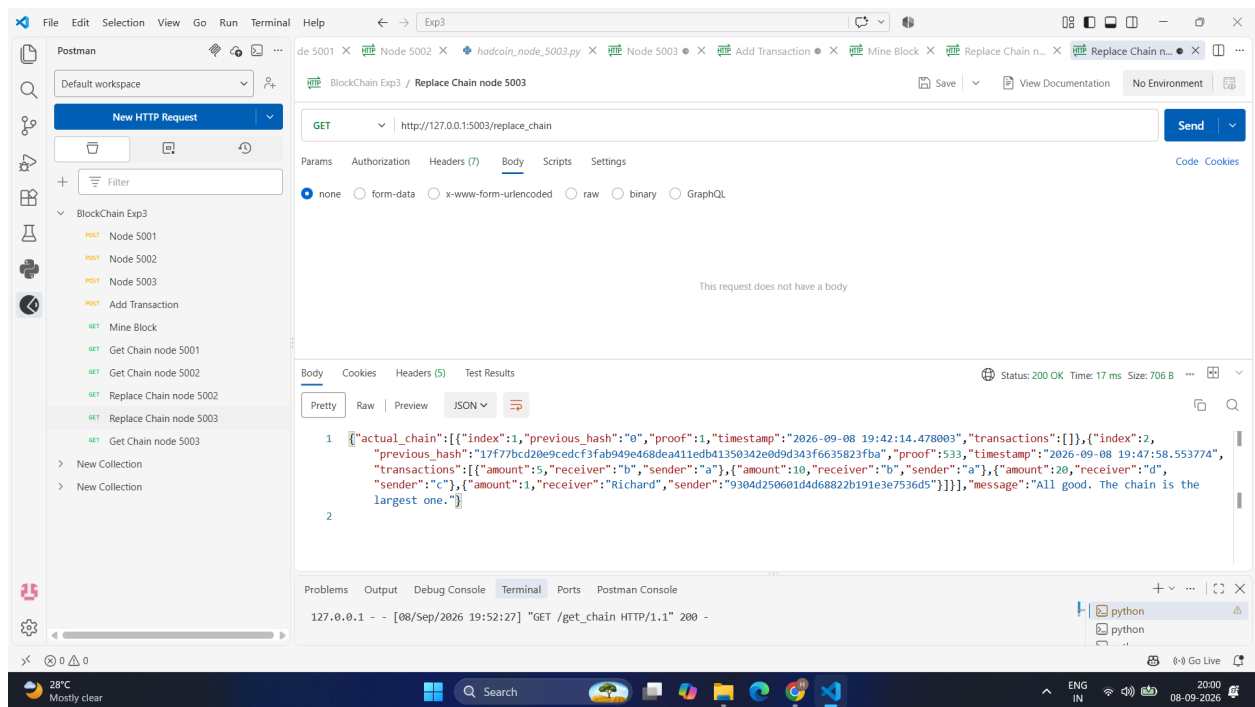
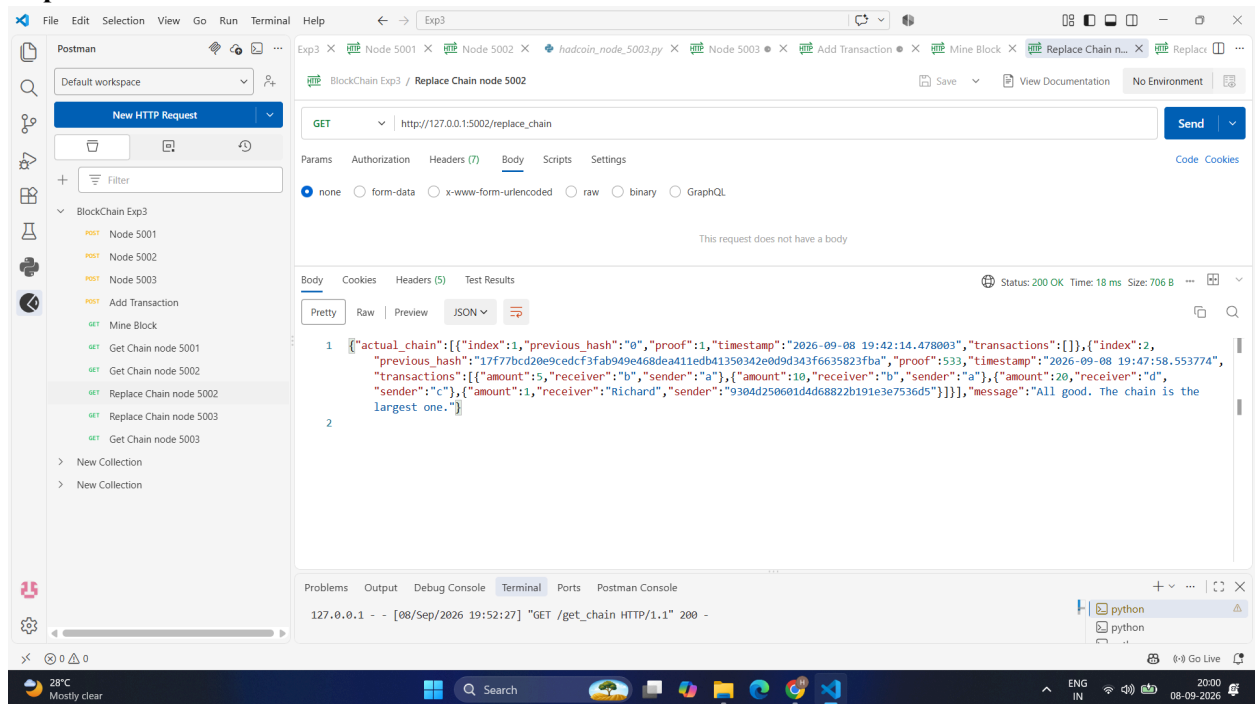
Step 3 : Transaction added



Step 4 : Mining



Step 5 : BlockChain



Conclusion:

Thus, the experiment demonstrates the practical implementation of important blockchain concepts such as cryptocurrency transactions, mining, Proof-of-Work, P2P networking, consensus, mempools, and blockchain synchronization using Python. The three-node setup provides a simple simulation of how independent blockchain nodes communicate and maintain a consistent distributed ledger.